CS 344: Algorithms **Rutgers: Spring 2026**

Homework #2

Name(s): $FIRST_1$ $LAST_1$, $FIRST_2$ $LAST_2$, $FIRST_3$ $LAST_3$

Homework Policy

- You may consult all course materials (lecture notes, textbook, slides, etc.) while writing your solutions. No external resources are permitted.
- **All submissions should be typeset in LaTeX, handwritten solutions are not accepted**.
- Remember to always **prove the correctness** of your algorithms and **analyze their running time** (or any other efficiency measure asked in the question).
- Homework assignments are completed and submitted in fixed groups of 3 students. Only **one member of the group submits** the full solution on Canvas. The submission must include the names of all group members. The other two members do **not need to submit anything separately**.
- You may discuss the problems with any of your classmates, even those outside your group. However, **each group must write and submit their solutions independently**.
- Attempt the challenge problems only if you find them interesting.

Acknowledgment (to be reviewed and edited as needed)

Our team members, **Leonhard Euler (le123)**, **Emmy Noether (en123)**, and **Srinivasa Ramanujan (sr123)**, collaborated extensively on this work. We dedicated 123 hours to independently analyzing Problem 17 before consulting external sources. After investing 123 days in Problem 32(c), we sought clarification from ChatGPT. Despite persistent efforts spanning 123 years, we were unable to solve Problem 49. **We hereby affirm that all external assistance utilized in the preparation of this document has been properly acknowledged**.

Problem 1. You are given an array $A[1..n]$ with the following properties:

- $A[1] > A[2]$ (the first element is greater than the second).
- $A[n-1] \leq A[n]$ (the second-to-last element is less than or equal to the last).

A **local minimum** is an element $A[x]$ such that:

$$A[x-1] \geq A[x] \leq A[x+1].$$

For example, in the array $A = [9, 7, 7, 2, 1, 3, 7, 5, 4, 7, 3, 3, 4, 8, 6, 9, 6]$, there are six local minima.

Design an $O(\log n)$-time algorithm to find a local minimum.

Problem 2. Suppose you are given two sorted arrays $A[1..n]$ and $B[1..n]$ containing distinct integers. Describe a $O(\log n)$ algorithm to find the median (meaning the nth smallest element) of the union $A \cup B$. For example, given the input

$$A[1..8] = [0, 1, 6, 9, 12, 13, 18, 20] \quad B[1..8] = [2, 4, 5, 8, 17, 19, 21, 23]$$

your algorithm should return the integer 9. [Hint: What can you learn by comparing one element of A with one element of B ?]

Problem 3. Write an algorithm in pseudocode:

- Input: An array A with n distinct (non-equal) elements
- Output-1: numbers x and y in A that minimize $|x - y|$, where $|x - y|$ denotes absolute-value(x-y). (If there are multiple closest pairs, you only have to return one of them.)
- Output-2: a pair of numbers w and z in A that minimize $|w + z|$.

The run-time should be significantly better than $O(n^2)$.

Problem 4. An inversion (swap) in an array $A[1..n]$ is a pair of indices (i, j) such that $i < j$ and $A[i] > A[j]$. The number of inversions in an n-element array is between 0 (if the array is sorted) and $\binom{n}{2}$ (if the array is sorted backward). Describe and analyze an algorithm to count the number of inversions in an n-element array in $O(n \log n)$ time. [Hint: Modify mergesort.]

Problem 5. Given an array A of length n, and two indices i and j with $i \leq j$, define $\text{Prod}(A, i, j)$ as the product of elements from $A[i]$ to $A[j]$, i.e., $A[i] \times A[i+1] \times \cdots \times A[j]$. If $i = j$, then $\text{Prod}(i, j) = A[i]$.

For example, if $A = [3, 0.1, 5, 20, 4]$, then $\text{Prod}(A, 1, 3) = 0.1 \times 5 \times 20 = 10$.

Now, consider the following problem MaxProd(A):

- **Input**: An array A of length n where all elements are positive, but may include fractional values (e.g., $A[i] = 0.1$ is valid).
- **Output**: The maximum possible value of $\text{Prod}(A, i, j)$ for some subarray $A[i..j]$.

For example, if $A = [3, 0.2, 4, 0.5, 1, 4, 0.3, 2]$, the maximum product is 8, which is achieved by the subarray $[4, 0.5, 1, 4]$.

a) Write pseudocode for a recursive algorithm $\text{MaxProd}(A)$ that solves the problem in $O(n \log n)$ time. You only need to provide the pseudocode and the recurrence relation for the algorithm. **(10 points)**

   Hint: The recurrence formula is similar to one we've used before, so you don't need to prove that it solves to $T(n) = O(n \log n)$.

   Note: While there are faster, non-recursive algorithms for this problem, you must use a recursive algorithm for this homework.

b) Now, write pseudocode for a recursive algorithm that solves the problem in $O(n)$ time. As in Part 1, provide the pseudocode and the recurrence relation for the algorithm. **(15 points)**

Hint: Similar to the Max Profit problem covered in class, solve a helper problem MaxProdX(i, j), which gives additional information beyond just the maximum product.

If you're confident with the $O(n)$ algorithm, you can directly write a single solution for both Part 1 and Part 2. However, if you're unsure, first implement the $O(n \log n)$ solution for Part 2, then write the separate $O(n)$ algorithm.

Problem 6. You are given two strings A and B of length n, each consisting of possibly repeated characters from an alphabet of size M. Two strings A and B are said to be **anagrams** of one another if it is possible to reorder the characters in A to obtain string B (without removing or adding any characters). For instance, the strings "ELEVEN PLUS TWO" and "TWELVE PLUS ONE" are anagrams of one another (think of empty-space also as a character in the alphabet).

Your goal is to design an algorithm that decides if a given pair of strings are anagrams of one another. You may assume that all characters in the alphabet can represented by distinct integers in the set $\{1, 2, \ldots, M\}$, and that you can convert any character to its integer representation in $O(1)$ time (note that M can be much larger than n).

(a) Design an algorithm for this problem with worst-case runtime of $O(n + M)$. **(15 points)**

(b) Design a randomized algorithm for this problem with worst-case expected runtime of $O(n)$.

Problem 7. Suppose we have an array $A[1 : n]$ which consists of numbers $\{1, \ldots, n\}$ written in some arbitrary order (this means that A is a *permutation* of the set $\{1, \ldots, n\}$). Our goal in this problem is to design a very fast randomized algorithm that can find an index i in this array such that $A[i] \mod 5 = 0$, i.e., $A[i]$ is divisible by five. For simplicity, in the following, we assume that n itself is a multiple of 5 and is at least 5 (so a correct answer always exist).

For instance, if $n = 10$ and the array is $A = [8, 7, 2, 5, 4, 6, 3, 1, 10, 9]$, we can output either of indices 4 or 9.

(a) Suppose we sample an index i from $\{1, \ldots, n\}$ uniformly at random. What is the probability that i is a correct answer, i.e., $A[i] \mod 5 = 0$? **(5 points)**

(b) Suppose we sample m indices from $\{1, \ldots, n\}$ uniformly at random and with repetition. What is the probability that none of these indices is a correct answer? **(5 points)**

Now, consider the following simple algorithm for this problem:

Find-Index-1$(A[1 : n])$**:**

- Let $i = 1$. While $A[i] \mod 5 \neq 0$, sample $i \in \{1, \ldots, n\}$ uniformly at random. Output i.

The proof of correctness of this algorithm is straightforward and we skip it in this question.

(c) What is the worst-case **expected** running time of **Find-Index-1**$(A[1 : n])$? Remember to prove your answer formally. **(7 points)**

The problem with **Find-Index-1** is that in the worst-case (and not in expectation), it may actually never terminate! For this reason, let us consider a simple modification to this algorithm as follows.

Find-Index-2$(A[1 : n])$**:**

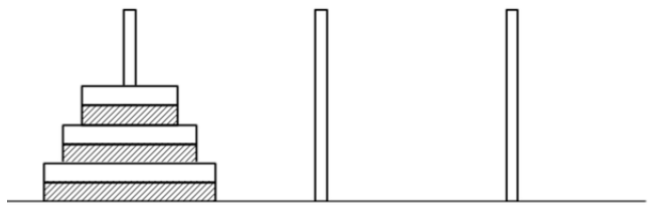
- For $j = 1$ to n:
  - Sample $i \in \{1, \ldots, n\}$ uniformly at random and if $A[i] \mod 5 = 0$, output i and terminate; otherwise, continue.
- If the for-loop never terminated, go over the array A one element at a time to find an index i with $A[i] \mod 5 = 0$ and output it as the answer.

Again, we skip the proof of correctness of this algorithm.

(d) What is the **worst-case running time** of **Find-Index-2**$(A[1 : n])$? What about its worst-case **expected** running time? Remember to prove your answer formally.

Challenge Yourself. Following are some variation of the Tower of Hanoi problem:

1. If all moves must be between adjacent pegs, what is the number of movies required? That is you cannot directive move from 1 to 3 for instance.
2. Now, consider a variation where for each disk size k, there are two identical disks. The goal remains to move all disks from peg 1 to peg 3 while following the standard Tower of Hanoi rules.



- If we ignore the fact that disks of the same size are identical and treat them as distinct disks, we can simply apply the standard Tower of Hanoi solution to $2n$ disks. What is the recurrence relation and the resulting number of moves?
- If we move both disks of the same size simultaneously while using standard Hanoi's algorithm, we are close to the solution. This strategy may flip the order of some equal size disks. Repeat this again to restore the order of disks. How many moves does this take?
- What is the optimal strategy to minimize the number of moves while preserving the original order of the disks?